

Resume Screening RAG Pipeline

Semantic resume retrieval, RAG Fusion, deterministic scoring, and grounded candidate explanations for hiring teams.

Team Name: ResumeRAG

Team Leader: Saurav Satpute

GitHub: github.com/sgsatpute/Resume-Screening-RAG-Pipeline

1,000

Resume corpus

synthetic resumes indexed

500

Test jobs

sampled JDs evaluated

0.945

Faithfulness

GPT-4 + RAG Fusion mean

0.843

Context recall

retrieved evidence coverage

Submission deck and ranked output generated from the local repository state.

The solution replaces brittle keyword screening with grounded candidate discovery.

Traditional ATS matching can miss semantically relevant resumes and over-reward keyword stuffing. This pipeline retrieves resume evidence first, then asks the LLM to summarize only the grounded shortlist.

Traditional matching

keyword overlap

static rules

weak context

- Exact word dependence misses equivalent terms.
- Long JDs are flattened into simple filters.
- Explanations can drift away from resume facts.
- Suspicious or low-quality profiles are not surfaced.

Proposed RAG approach

semantic FAISS

RAG Fusion

grounded LLM

- Sentence-transformer embeddings retrieve similar resumes.
- Sub-queries cover different JD requirements.
- Final score combines words, phrases, and skills.
- Output includes evidence and data-quality flags.

Differentiator: the LLM explains a ranked evidence pack; it does not independently invent or reorder candidates.

A JD is converted into a ranked shortlist through five auditable stages.

Each stage produces a concrete intermediate artifact: query type, sub-queries, candidate IDs, scored resume contexts, and a streamed recruiter-facing answer.



Trace output

The verbosity panel exposes query type, selected RAG mode, generated sub-queries, retrieved resume IDs, RRF scores, and elapsed time. This makes debugging and challenge review repeatable.

Candidate fit is evaluated as a bundle of role, skill, phrase, and evidence signals.

The implementation keeps JD parsing lightweight and deterministic, then lets semantic retrieval handle natural-language variation.

Role intent

developer, engineer, analyst,
designer, manager, role, job

Hard skills

Python, Java, React, SQL, ML,
TensorFlow, AWS, Docker,
Scrum

Experience

years of experience, seniority
terms, hands-on delivery
language

Domain/tools

frameworks, databases,
cloud, DevOps, product area,
industry context

Soft signals

communication,
collaboration, ownership,
problem solving, availability

Beyond keyword matching

FAISS captures semantic similarity, bigrams preserve phrases such as machine learning, skill density breaks ties, and evidence snippets keep the recommendation auditable.

semantic match

phrase match

skill density

The final ranking blends semantic retrieval with transparent presentation scoring.

FAISS finds candidates; reciprocal-rank fusion merges sub-query lists; compute_score makes the displayed order interpretable.

1. Semantic candidates

`similarity_search_with_score(question, k=5)`

2. RAG Fusion merge

RRF adds $1 / (\text{rank} + 50)$ across original and generated sub-queries

3. Full resume mapping

candidate IDs are mapped back to full resume text from the dataframe

4. Final presentation score

unigram overlap + 2x bigram overlap + up to 2 point skill-density bonus

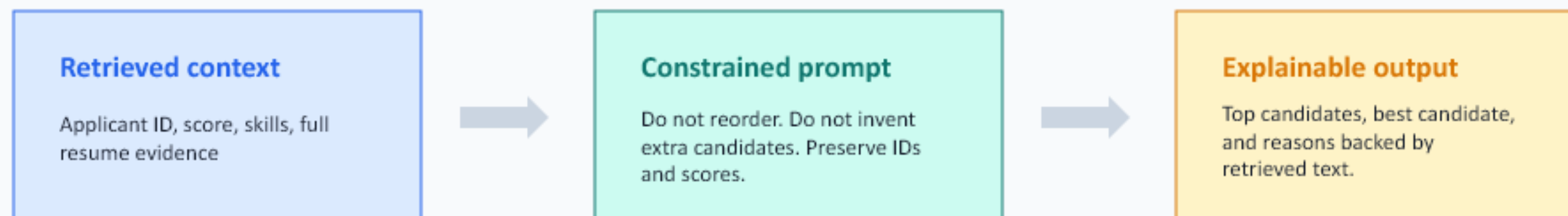
Final score formula

$((\text{unigram hits} + 2 \times \text{bigram hits}) / \text{query terms}) \times 100$
+ skill-density bonus

Why this matters: recruiters see a stable, explainable score while the vector index still handles semantic discovery.

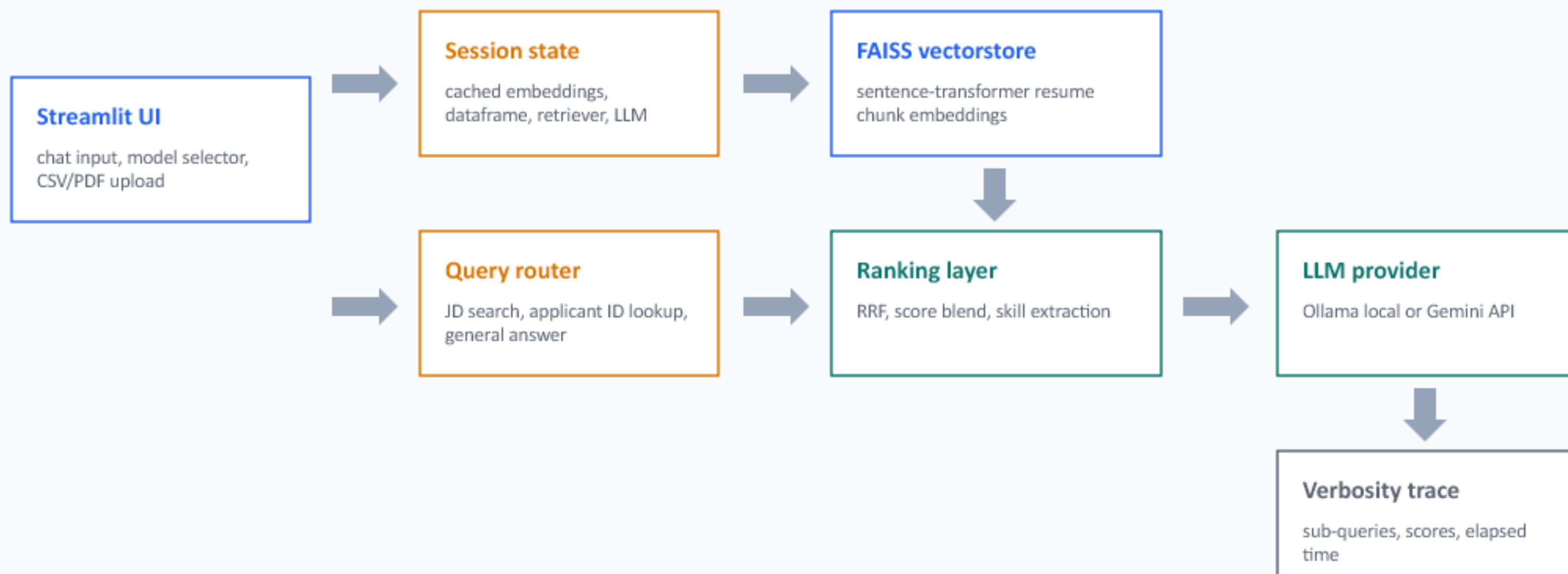
Explanations are grounded in retrieved resumes and guarded by explicit quality checks.

The LLM receives formatted candidate context with applicant ID, match score, skills, and resume text, then follows strict response constraints.



The architecture separates UI state, retrieval, scoring, and generation.

This keeps the expensive embedding/vectorstore objects cached while allowing user-controlled model and RAG-mode selection.



Evaluation outputs show strong grounding, and the stack stays laptop-friendly.

The best available run is GPT-4 + RAG Fusion over 500 evaluation rows; the runnable demo also supports fully local Ollama inference.

0.793

Context precision

mean over 500 rows

0.843

Context recall

mean over 500 rows

0.945

Faithfulness

mean over 500 rows

0.733

Answer similarity

mean over 500 rows

Runtime and compute fit

- FAISS enables fast local approximate nearest-neighbor search.
- all-MiniLM-L6-v2 keeps embeddings CPU-friendly.
- Streamlit caches session objects to avoid repeated model/index loads.
- Ollama option supports offline/local demo runs.

Technologies selected

Streamlit

FAISS

LangChain

Ollama

Gemini

sentence-transformers

Pandas

Submission assets: public GitHub repository, this deck exported to PDF, and ranked_candidate_output.xlsx.